UrbanGaon AI Todo Platform

Executive Architecture & System Engineering Manual

OFFICIAL RELEASE V2.4

Date: 30 October 2026
Author: Lead Architect & SDE-3

1. Executive Summary & Root Architecture

The UrbanGaon AI Todo Platform is an enterprise-grade deliverable coordination and operational governance operating system. It features natural language voice parsing, full calendar workflow management, automated morning email digests, nightly carry-forward rollovers, multi-assignee tracking, and real-time tab synchronization.

The codebase has been refactored into clean, isolated root modules:

```
d:\TODO Dashboard New updated Version\
├── app/                  # Next.js 15 App Router (Thin routing & endpoints only)
│   ├── layout.tsx, page.tsx
│   └── api/              # REST Endpoints (delegate to backend/)
├── frontend/             # Client-side presentation & UI state
│   ├── components/       # 15 React 19 UI Portals & Views
│   ├── context/          # AppContext: Global client state & BroadcastChannel
│   ├── hooks/            # Custom hooks (useTaskDeadlines)
│   ├── lib/              # Client utilities (sound.ts, exportUtils.ts)
│   └── styles/           # Design tokens (tokens.css) & globals.css
├── backend/              # Server infrastructure & business engines
│   ├── lib/              # db.ts, auth.ts, cron.ts, digestEngine.ts, overdueEngine.ts, etc.
│   └── models/           # Mongoose ODM schemas (Task, Member, User, AuditLog)
├── shared/               # Isomorphic logic (Universal between UI & Server)
│   ├── lib/              # dateUtils.ts, rosterData.ts, multiAssignee.ts, mockData.ts
│   └── types/            # TypeScript domain models & contracts
└── public/               # Pure RGBA transparent logos & favicons
```

2. Where is server.js & How the System Runs

In modern Next.js 15 App Router architecture, server execution depends on the target environment:

- **Development (`npm run dev`):** Next.js runs its built-in development HTTP engine with Fast Refresh, on-demand TypeScript compilation, and route compilation.
- **Containerized / Standalone (`npm run build`):** Using `output: 'standalone'` in `next.config.ts`, Next.js compiles a self-contained production bundle at:

  ```
  .next/standalone/server.js
  ```

  This is the exact entry point started by our production Docker container on the Synology DS925+ NAS:

  ```
  CMD ["node", "server.js"]
  ```

- **Vercel Serverless:** Each route handler under `app/api/*` is compiled into an isolated Edge/Node.js serverless lambda function that scales automatically.

3. Background Automated Engines (IST Timezone)

Server background schedulers are booted via `instrumentation.ts` when the Node process starts. All jobs execute strictly in the `Asia/Kolkata` timezone:

Engine	Schedule (IST)	File Location	Operational Responsibilities
Morning Email Digest	08:30 AM Daily	`backend/lib/digestEngine.ts`	Evaluates each employee's overdue, due today, and upcoming tasks. Dispatches HTML digest via Nodemailer SMTP.
Midnight Rollover	00:05 AM Daily	`backend/lib/cron.ts`	Carries forward all uncompleted tasks with `scheduledDate < today` to today's date. Never modifies the committed `dueDate`.
Hourly Overdue Governance	Every Hour (at :00)	`backend/lib/overdueEngine.ts`	Flags expired deliverables, calculates days overdue, demands justification reasons, and auto-escalates to managers at 3+ days.
Data Retention Engine	02:00 AM Daily	`backend/lib/retentionEngine.ts`	Archives completed tasks older than 365 days and audit logs older than 730 days into summary collections.

4. End-to-End Data Lifecycle Flow

1. **Voice & Input Processing:** Natural language phrases are parsed in real time by `shared/lib/aiParser.ts`, automatically extracting the task title, priority, target deadline, and assigned staff members.
2. **API Dispatch & Validation:** Client components send a `POST /api/tasks` request. The route handler validates permissions, applies role-based access control (RBAC), and saves the deliverable to MongoDB Atlas via `TaskModel`.
3. **Audit Logging & Notifications:** An immutable record is created in `AuditLogModel`, and transactional email notifications are sent to assignees via `backend/lib/notificationDispatcher.ts`.
4. **Cross-Tab Client Synchronization:** The browser's `BroadcastChannel('urbangaon_task_sync')` broadcasts the update, causing all other open tabs to synchronize their React state instantly without page reloads.

> **✓ Quality & Verification Guarantee**
> All 26 Next.js routes, server components, and database models have passed strict TypeScript validation (`npx tsc --noEmit`) with **0 errors** and built cleanly into production standalone artifacts.